Regression and classification analysis with neural networks

Amund Rekkavik, Kaja Sandnes
Department of Physics, University of Oslo
(Dated: November 9, 2026)

Neural networks can be used to solve a broad variety of problems, regression and classification among them. In this paper we train a multilayer perceptron neural network to fit the Runge function using the backpropagation algorithm together with stochastic gradient descent. By tuning the learning rate and regularization term as well as optimizing the width and the depth for the sigmoid, ReLU and Leaky ReLU activation functions, we obtain a mean squared error comparable to that of linear regression with OLS. By changing the final activation and cost function of the network to softmax and cross entropy, we train an MLP with a single hidden layer on the classification task of the MNIST handwritten digits dataset. By minimizing 5-fold cross-validated accuracy on the training set by optimizing learning rate, number of epochs, the width of the hidden layer, and L^2 regularization strength, we achieve a final test set accuracy of $98.1\% \pm 0.1\%$, significantly outperforming a logistic regression baseline.

I. INTRODUCTION

Regression and classification problems are the first problems one encounters when studying supervised learning. The underlying principle is to give a model a suitable amount of freedom in processing an input, and train its parameters until it reaches a satisfactory error or accuracy. Linear regression models handle regression, and logistic regression models handle classification. But these models have a problem. There are problems they simply cannot solve. Namely, problems with *non-linear* decision boundaries [1]. The solution is to add a *hidden layer* to our model. This hidden layer must be accompanied by a non-linear activation function, otherwise the linear transformations simply form another linear transformation through matrix composition, and we are back to square one. Letting A, B be linear operators, $\mathbf{b}$ a bias, and $\mathbf{x}$ our input

$$A(B\mathbf{x} + \mathbf{b}) = Cx + \mathbf{c} \tag{1}$$

Where $\mathbf{c} = A\mathbf{b}$, and $C = AB$. Inserting instead our non-linear activation function:

$$A\sigma(B\mathbf{x} + \mathbf{b})$$

And the expression is irreducible. We have built a *multilayer perceptron* (MLP) model [1].

In this project we will use neural networks to solve regression and classification problems. The regression problem we will be solving is that of approximating the Runge function, famously unstable at high model complexities. We will be comparing the performance of a multilayer perceptron model on this problem to that of linear regression models. The classification problem we will be tackling is that of the MNIST handwritten digit dataset, a common benchmark for classification models. We will be using the training set to find ideal hyperparameters for the model through cross-validation, and we will compare its performance to logistic regression.

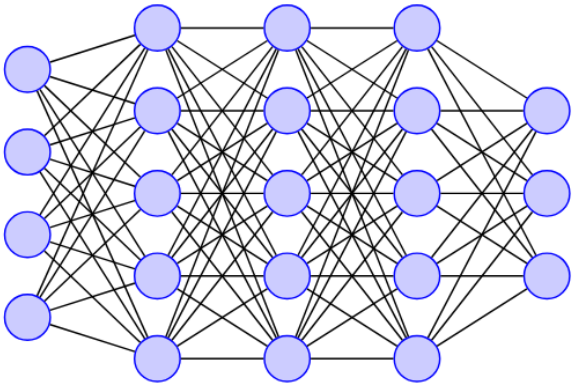


Figure 1. An illustration of a fully connected neural network with three hidden layers. Inputs travel left-to-right and change dimensionality between layers. Hidden layers are connected with non-linear activation functions.

II. THEORY

A. Analytical gradients

Neural networks learn through gradient descent. It is therefore extremely important to know the gradients of the cost- and activation-functions used in these networks. The final activation, $\tilde{y}$, which is inserted into the cost function depends nonlinearly on all the weights and biases in a fully connected network. Gradients must therefor be calculated through the repeated application of the chain rule. Either analytically, or through automatic differentiation. This process is called *backpropagation*. We will now take a look at different gradients which help us do backpropagation.

The mean squared error, or MSE, takes on the form:

$$\mathcal{L} = MSE(\tilde{y}, y, \boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=1}^{n}(\tilde{y}_i - y_i)^2 \tag{2}$$

We can find it's derivative with respect to the final activation through one application of the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \tilde{y}} = \frac{2}{n}(\tilde{y}_i - y_i) \tag{3}$$

The MSE is the standard cost function when solving regression problems.

When solving classification problems, however, we tend to use *cross entropy*. Cross entropy takes on two forms: *binary* cross entropy, and *multiclass* cross entropy. These are used when the number of classes is exactly two, or greater than two. These cost functions are always used in conjunction with the sigmoid, and cross entropy functions as final activation functions. Why? That we shall see shortly. First the binary case. We have the binary cross entropy:

$$CE = -\sum_{i=1}^{n} y_i \log \tilde{y}_i + (1 - y_i)\log(1 - \tilde{y}_i) \tag{4}$$

As our cost, and the sigmoid:

$$\sigma(z_i) = \frac{1}{1 + e^{-z_i}} \tag{5}$$

As our final activation. We can compute their derivatives:

$$\frac{\partial CE}{\partial y_i} = -\frac{y_i}{\tilde{y}_i} + \frac{1 - y_i}{1 - \tilde{y}_i}, \quad \frac{\partial \sigma(z_i)}{\partial z_i} = \sigma(z_i)(1 - \sigma(z_i)) \tag{6}$$

Using the chain rule, we multiply them together to find:

$$\frac{\partial CE}{\partial z_i} = \tilde{y}_i - y_i \tag{7}$$

It simply reduces to their difference. This does, in fact, also happen to the multiclass cross entropy and the softmax!

$$CE = -\sum_{i=1}^{n} y_i \log \tilde{y}_i \tag{8}$$

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_k e^{z_k}} \tag{9}$$

Applying the chain rule gives, again:

$$\frac{\partial CE}{\partial z_i} = \tilde{y}_i - y_i \tag{10}$$

If we apply regularization, the cost gets two extra terms:

$$\mathcal{L} = C(\tilde{y}, y, \boldsymbol{\Theta}) + \lambda_1 \sum |w_{i,j}^l| + \frac{\lambda_2}{2}\sum (w_{i,j}^l)^2 \tag{11}$$

Taking the derivative with respect to one of the weights:

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^l} = \frac{\partial C}{\partial \tilde{y}}\frac{\partial \tilde{y}}{\partial w_{i,j}^l} + \lambda_1 \operatorname{sign}[w_{i,j}^l] + \lambda_2 w_{i,j}^l \tag{12}$$

Note that the L^1 norm is *not differentiable* when its argument is zero as the sign operator is not defined for zero.

There are, in addition, two other activation functions we would like to discuss. These are the "rectified linear unit", or ReLU. And the so-called "leaky ReLU". First, the piecewise linear ReLU function takes on the form [1] (eq. 8.23):

$$ReLU : \mathbb{R} \to \mathbb{R}^+, \quad ReLU(x) = \max(0, x) \tag{13}$$

The function is the linear identity for positive x, and simply maps all negatives to zero. This gives it a derivative of:

$$\frac{dReLU(x)}{dx} = H(x) \tag{14}$$

Where H is the *Heaviside step function*. Exactly how this function should be defined at $x = 0$ is up to convention in the field, as the ReLU function is not differentiable there. We have defined it as:

$$H(x) = \begin{cases} 1, & x > 0 \\ 0, & \text{else} \end{cases} \tag{15}$$

As ReLU sets the gradient to zero for all inputs smaller than or equal to 0, it can lead to the "dying ReLU". Which is a characteristic failure of the function. If a unit outputs negative values for every input, ReLU sets all of its outputs to zero. The gradient is then set to zero, and the unit's weights never change [1]. One way to fix this is to modify the function in a way to keep the underlying idea of flattening it below zero, but in such a way that its gradients don't completely vanish. This is done with the "leaky ReLU" function [1] (eq. 8.24):

$$LReLU : \mathbb{R} \to \mathbb{R}, \quad LReLU(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \tag{16}$$

For some small trained parameter $\alpha \approx 0.01$. This function is not differentiable either. In our project we use the following convention for the derivative:

$$\frac{dLReLU(x)}{dx} = \begin{cases} 1, & x > 0 \\ \alpha, & x \leq 0 \end{cases} \tag{17}$$

B. Binomial standard error and confidence

For the classification analysis we are going to be needing some statistics of binomial distributions. If our underlying distribution is of a binary random variable and X is the number of successes in n trials with a probability p for success: [2]:

$$X \sim \text{binom}(n, p) \tag{18}$$

For our classification models, we can think of this as the number of datapoints correctly classified on a validation set. Our accuracy is then the same as the unbiased estimator for the underlying probability:

$$\hat{p} = \frac{1}{n}X = \frac{1}{n}\sum_{i=1}^{n} I(\tilde{y}_i = y_i) = \text{Accuracy} \tag{19}$$

Where I is the indicator-function. The standard error of this estimator is [2]:

$$\hat{s} = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \tag{20}$$

To calculate a confidence interval, we than simply use:

$$\hat{p} \pm z_{\alpha/2}\hat{s} \tag{21}$$

For the classification analysis we will be using the standard 95% confidence interval. Using the normal approximation, we get:

$$\hat{p} \pm 1.96\hat{s} \tag{22}$$

Note that this approximation is valid in this case, as the binomial distribution grows toward a normal distribution as $n \to \infty$, and we are dealing with sufficiently large datasets for classification.

III. METHOD

A. Implementing feed-forward neural network

Our network carry information in one direction only, which makes it a feed-forward network. It is also a fully connected network, meaning that each neuron receives a weighed sum of the output activations of all neurons in the previous layer. Each neuron has its own activation function $\sigma(z) = a$ that transforms the weighted sum z nonlinearly into the activation output a, which is fed to all neurons in the next layer. The depth of the network refers to the number of hidden layers, and the width refers to the number of nodes in each layer.

When initializing the network, we give it a list of numbers that tells how many neurons there are in each layer. Each node is associated with a bias b and connects to and associates an unique weight w with each of the nodes in the previous layer. Correct initializations of these weights and biases can speed up learning and reduce the risk of saturating gradients. The weights are drawn from a normal distribution with mean 0. The variance of the distribution depends on the activation function associated with the neuron. Our network use two types of weight initialization, Xavier (Glorot) initialization and He initialization, which serve different activation functions. Let M_l be the number of nodes in layer l, then we have the relation

$$Var(z) = M_{l-1}Var(w)Var(a) \tag{23}$$

Optimally, we want the variance of the weighted activation $Var(z)$ to equal the variance of the activations from the previous layer $Var(a)$. For this to be true when going backward and forward, we set the number of nodes to be the average of the past and the current layer, such that the variance of the weights are

$$Var(w) = \frac{2}{M_{l-1} + M_l} \tag{24}$$

This is Xavier initialization, which works well for the sigmoid activation function. Because normal distributed weights brake the symmetry between the nodes in each layer, the biases are initialized to 0. However, for activation functions in the ReLU family, which maps all negative values to 0, half of the nodes are inactive in the forward pass, and therefore halves the variance. Therefore weights are initialized with the variance

$$Var(w) = \frac{2}{M_{l-1}} \tag{25}$$

Because the ReLU gradient is zero for negative values, the biases are initialized to 0.01, to ensure that the nodes still learn.

Just like with linear regression, the goal of the neural network is to find the parameter vector $\boldsymbol{\Theta} = \{\boldsymbol{W}^l, \boldsymbol{b}^l\}_{l=1}^{L}$ that minimizes the cost function

$$\boldsymbol{\Theta} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}) \tag{26}$$

Here $\boldsymbol{W}^l$ is the matrix of all weights in layer l, where each element w_{ij}^l is the weight associated to the connection between the i'th node in layer $l-1$ and the j'th node in layer l. Our network uses gradient descent to update the weights and biases. To calculate the gradients and the errors, our network uses a backpropagation algorithm consisting of these four

equations

$$\boldsymbol{\delta}^L = \nabla_a C \odot \sigma(\boldsymbol{z}^L) \tag{27}$$

$$\boldsymbol{\delta}^l = ((\boldsymbol{W^{l+1}})^T \boldsymbol{\delta^{l+1}}) \odot \sigma'(\boldsymbol{z}^l) \tag{28}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{29}$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1}\delta_j^l \tag{30}$$

For derivations of these four equations, see [3]. Backpropagation is an algorithm that computes the gradient of the cost function with respect to every weight and bias, and gives detailed information about how the network learn. Firstly, the network sends the input data through all hidden layers such that the output layer L returns $\boldsymbol{a}^L = \sigma(\boldsymbol{z}^L)$. On its way from the input to the output layer, all activations $\boldsymbol{a}^l$ and weighted activations at each layer $\boldsymbol{z}^l$ is saved. Eq. 27 calculates, which we saw in the theory section from Eq. 7 with a clever choice of cost- and activation function, the total error made by the network. Errors in previous layers are then calculated by Eq. 28. At last, the gradients are calculated by Eq. 29 and 30. We also implemented code to compare the backpropagation gradients against automatic differentiation by jax [4].

B. Runge data regression analysis

The input data X was $n = 100$ equally spaced data points in the range $[-1, 1]$. The target y was generated by sending X through the Runge function

$$f(x) = \frac{1}{1+25x^2} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mu, \sigma^2) \tag{31}$$

where noise was generated by a normal distribution $\mathcal{N}$ with mean $\mu = 0$ and variance $\sigma^2 = 0.1$, using NumPy's random library with seed 2026. Both X and y was reshaped to matrices with shape $(n, 1)$, and split into training and test data, the latter of size 20%, using scikit-learn's train_test_split function [5]. The input data was then scaled using scikit-learn's transform function, and the target centered by subtracting the mean over all target data points from each target point. When using gradient descent methods, standardizing the data reduces the risk of saturating gradients which prevent the network from learn. Why it is important to standardize also follows the intuition from linear regression, where standardizing reduces the condition number.

The design matrix reduces to a single column consisting of the input data. We used the same data as in our previous project to be able to compare our network with the linear regression models. 20% of the training data was again split into validation data, used to tune hyperparameters, such that test data do not leak into the training. Because we only had 100 data points, where 20 of them was test data and 64 and 16 of them were training and validation data respectively, we used 5-fold cross validation to maximize profit from the data we have by systematically training and validating on different subsets of the data. The averaged MSE of the data tested against $k = 5$ different validation sets are calculated, and the uncertainty was found by taking the standard error of mean $SEM = \hat{\sigma}/\sqrt{k}$. This way we could find the best combination of learning rate and optimizer method from the selection $\gamma \in [001, 0.01, 0.1]$ and methods which were plain gradient descent and stochastic gradient descent with Adam or RMSprop. We tested the parameters both for a 50-node single hidden layer network and a double hidden layer network with 50 nodes each, 100 in total. We used the MSE Eq. 11 as cost function, the sigmoid function Eq. 5 as activation function for the hidden layers, thereby also the Xavier (Glorot) initialization Eq. 24, and the identity as output function at the output layer. In our case, we do not want to use the sigmoid function as output function because it maps all numbers to $(0, 1)$ when we want the data to be mapped to any possible output value the Runge function can take.

After finding the combination of learning rate and optimizer method which gave lowest MSE, we used this parameters to explore the network architectures and find the optimal width and depth of the network. We tried out 1, 2, 3 and 4 hidden layers in combination with 10, 25, 50 and 100 nodes per layer for three different activation functions, sigmoid, ReLU and Leaky ReLU. The sigmoid function is the same as we have already used, the ReLU activation Eq. 13 and its derivative Eq. 15 use He initialization Eq. 25, same as when using Leaky ReLU activation Eq. 16 which has the derivative 17. All three uses the MSE as cost function and the identity as output function.

Then we added regularization terms to the cost function exclusively used in the backpropagation algorithm. We used a hyperparameter λ to tune the regularization. The regularization was only applied to the weights, and is for the network to prefer smaller weights to avoid overfitting. We do not want to penalize the biases because they play the role of the intercept, which is the output value when the input values are zero. The regularization terms used

are the following:

$$C_{L2} = MSE + \frac{\lambda}{2}\sum_{l=1}^{L}\|W^l\|_2^2 \tag{32}$$

$$C_{L1} = MSE + \lambda\sum_{l=1}^{L}\|W^l\|_1 \tag{33}$$

At last, after obtaining the optimal parameters for the different activations: sigmoid, ReLU, Leaky ReLU, ReLU with L1 and L2 regularization, we made a final fit for all of them and calculated the optimal MSE for each method, which was compared to correspondingly to OLS, Ridge and Lasso.

C. MNIST dataset classification analysis

The MNIST dataset is one of the most famous benchmark datasets for classification models. The dataset consists of 70000 images of 28×28 pixels, split into canonical test and training sets containing 60000 and 10000 images respectively [6]. The pixels in the images are grayscale, so all the information in each pixel is contained in one number in the range $[0, 255]$ deciding its colour-value. After the data was loaded, it was scaled to unit dimensionality, and each of the images were flattened to vectors with 784 components. Now the data was on a form that our MLP network could work with.

Our goal was to find the optimal hyperparameters for our classification model with the training data. In the classification case, we optimize for accuracy, which in this case, is simply the fraction of images correctly classified. In order to find the optimal hyperparameters, we employed 5-fold cross validation on the training set. Now, there are simply too many hyperparameters to tune in a neural network to investigate all of them with limited time and computational power. We have limited ourselves to tuning four parameters, two of which having to do with the architecture of the model, and two of which having to do with the learning process. These parameters are:

- The learning rate: γ
- The number of epochs in the SGD
- The number of hidden nodes in the hidden layer
- The regularization strength: λ

The rest of the hyperparameters have been chosen beforehand. What they are, and our reasons for picking them will be found in the section on the classification model.

1. The classification model

Our model takes the form of a fully connected feed-forward neural network with one dense hidden layer of width n. We have restricted ourselves to one hidden layer, as tuning every other parameter together with many different numbers of layers would be computationally expensive, and as the addition of a single hidden layer should give our network the non-linearity it needs to be able to outperform a logistic regression model. We wish to see how well we can beat it. The activation function of the hidden layer was chosen to be ReLU. For classification problems, ReLU is the standard activation. As discussed in the theory section, it has issues such as the dying ReLU. Given more time we might have tried the leaky ReLU, or other activations i the ReLU family in addition, but we should be able to achieve our goals with ReLU. Again, trying many different activation functions for every model architecture would have been too time-consuming. The output activation is the softmax function. Together with the multiclass cross entropy as a cost function, it makes for the ideal output for a classification model. The training is done with stochastic gradient descent to optimize run time, and with the Adam optimizer of standard Optax schedule ($\beta_1 = 0.9, \beta_2 = 0.999$). This is standard procedure for the training of neural networks. During training, a batch size of 256 was used to get a large enough sample of the large training set, but to still reap the rewards of the stochastic gradient descent. Our model therefore takes on the form:

$$x \to (784, n) \to ReLU \to (n, 10) \to \sigma \to \tilde{y}$$

Where n is the width of the hidden layer, and σ is the softmax function. The cost function takes on the form:

$$\mathcal{L} = \sum y_i \log \tilde{y}_i + \frac{\lambda}{2}\sum (w_{ij}^l)^2 \tag{34}$$

We have chosen to apply L^2 regularization of strength λ. The L^2 norm was chosen as it is simpler to work with than the L^1 norm is. Due to smooth gradients, training will be more consistent. Lastly, the weights of the model were initialized according to He-initialization [1] (eq. 8.50):

$$w_{ij}^l \sim \mathcal{N}(0, \sigma^2), \quad \sigma^2 = \frac{2}{n_{in}^l} \tag{35}$$

This initialization works well for networks with ReLU activation in their hidden layers[1]. Biases were initialized to 0.01 to avoid ReLU death.

2. *The cross-validation*

As mentioned above, 5-fold cross validation was employed to select hyperparameters. Here, a subclass of the NeuralNetwork class was created, that also has inheritance from the scikitlearn abstract base class: "BaseEstimator" in order to use sklearn cross validation functionality. Sklearn's stratified k-fold was used, in order to equip each of the splits with an equal number of classes to ensure reliable training. In accordance with cross-validation, the training set was split into 5 different sets, each with 12000 images. Models were trained on four of these sets, and then validated on the last one five times per combination of hyperparameters. This gives us an estimate for accuracy and standard error, taken as the mean accuracy and its variance over folds. The cross validation was conducted on grid searches, where a range of parameters is selected, and models are trained and validated with these parameters. This way we can see how different combinations of parameters lead to differing performances. Optimal combinations are chosen with the one-standard-error-rule. This means that we find the best score in the grid search, and select the "simplest" model that scored within one standard error of the best score. For our network, a lower learning rate, fewer epochs, fewer hidden nodes, and stronger regularization all correspond to a simpler model.

IV. RESULTS AND DISCUSSION

A. Runge data regression analysis

Model	MSE (± noise)	HL x w	γ	λ	init
Sigmoid	0.008	1 x 10	0.01	0.0	Xavier
ReLU	0.011	1 x 25	0.01	0.0	He
LReLU	0.011	1 x 25	0.01	0.0	He
L_1	0.007	3 x 50	0.01	0.001	He
L_2	0.007	2 x 50	0.01	0.001	He
OLS	0.013 ± 0.001				
Ridge	0.013 ± 0.003			$6.2 \cdot 10^{-7}$	
Lasso	0.020 ± 0.004			$4.8 \cdot 10^{-5}$	
Noise floor	0.01				

Table I. Lowest MSE scored obtained with different activations. L_1 and L_2 use ReLU activation. OLS and Ridge had minimum MSE at $p = 13$. Lasso at $p = 8$. Noise floor is the variance of the target data.

In Table I we see that the lowest MSE for our sigmoid network and the regularized network has exceptional low MSE, smaller than the noise floor. Either the network was exceptionally well fitted with hyperparameters, or test data might have leaked into the training. Looking at Figures 2 and3 we see that the MSE calculated from validation data never goes beneath $MSE = 0.011$, with an uncertainty $SEM = 0.001$ that overlap with the OLS variance, which mean that they cannot be distinguished. Even when looking away from the exceptional low L_1 MSE in the table and rather looking at the values in Figure 4, the lowest $MSE = 0.011$ is a big improvement from the Lasso MSE obtained from linear regression. The corresponding value for L_2 is $MSE = 0.012 \pm 0.001$, which overlap with the Ridge regression. From these values we see that our neural network perform at the same accuracy, if not better than the linear regression models.



Figure 2. MSE of different learning rates in combination with different gradient descent methods with different optimizers.

In Figure 2 we see that stochastic gradient descent performed significantly better than plain gradient descent. Training the neural networks can take between few minutes to hours, so using stochastic gradient descent, which only calculate the gradient on a fraction of the data points at a time, may be more time efficient, though, for a network with only 80 training points, this might not be the main reason. However, by using the optimizers Adam and RMSprop, the learning rate is adapted during training, which makes the network learn faster. Even though the variances are large, Adam seems to generally obtain a lower MSE than RMSprop, and was therefore the estimator we continued to use when tuning other parameters.

In Figure 3 of the sigmoid architecture, we see a trend in increased MSE as the number of layers and

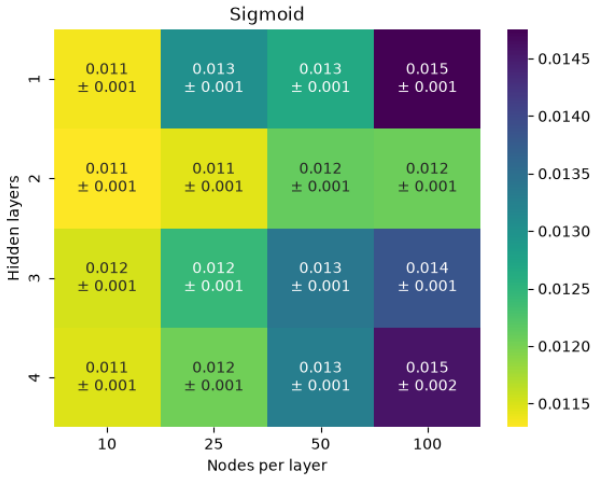


Figure 3. MSE plotted as a function of number of nodes and layers for the sigmoid activation function.

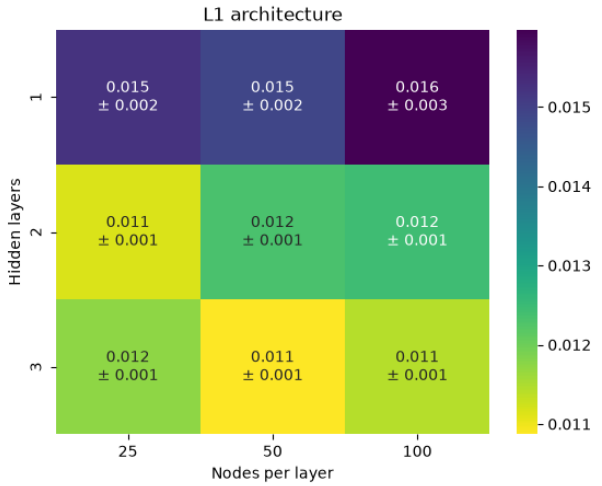


Figure 4. MSE of ReLU activation with L1 regularization plotted as a function of nodes per layer and number of hidden layers.

nodes grow. It is tempting to argue that the trend is due to a well known issue concerning vanishing gradients of the sigmoid function due to repeatedly multiplication of already small gradients in the back-propagation algorithm, which causes the gradients in the earliest layers saturate and effectively stop learning. However, by going down the layers in the 10 node column, the MSE values are completely fine. The issue seems to arise when increasing the number of nodes. Using a network with 100 nodes on a data set with 80 data points will clearly lead to overfitting. We can see in Table I that all the models without a regularization term, sigmoid, ReLU and Leaky ReLU, performed best with only one layer and few nodes. Only when the weights was penalized with the L_1 and L_2 norms, overfitting may be avoided, which is confirmed in Figure 4, because the MSE is decreasing rather than increasing as the complexity of the model increase.

B. MNIST dataset classification analysis

When selecting the optimal parameters through cross-validation, we used the grid-search method. We did 5-fold cross-validation, so the number of models to train grows quickly. Ideally, we could have chosen as many points as we wanted, and done a 4-dimensional grid-search of the parameters we wanted. This would, however have required us to train $5 \cdot n_p^4$ models, where n_p is the number of different values of parameters we have tried. We instead opted to grid search two and two parameters. We started with learning rate, and the width of the layer. We did this grid-search at a constant number of epochs: 10, and at no regularization. We grid-search these parameters together, as different model architectures require different training processes. And we search regularization with number of epochs, as it allows us to control possible overfitting due to long training. We tested $\gamma \in \{0.001, 0.01, 0.03\}$, and $n_{nodes} \in \{64, 128, 256, 512\}$. The resulting cross-validated accuracies and standard errors can be found in figure 5: We see that the one-standard-error rule doesn't

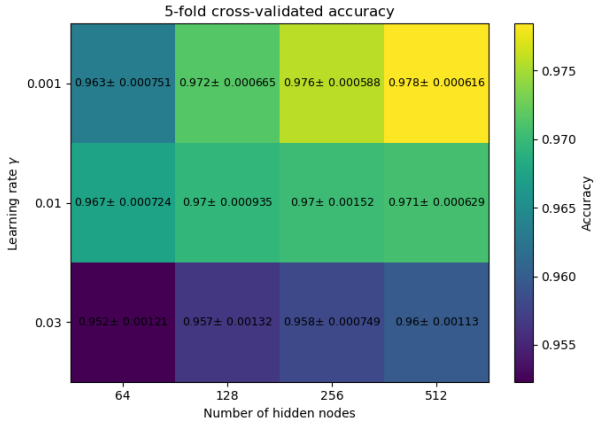


Figure 5. Heatmap of the learning rate / width grid-search. For model at 10 epochs and no regularization. Highest accuracy was achieved for $(\gamma = 10^{-3}, n = 512)$, distinguishable to one standard error.

contribute to our decision here, as the standard errors are too low. The clearly highest accuracy is gained from the lowest learning rate, combined with the widest hidden layer. These are then the learning rate and layer width we will be using when optimizing our number of epochs and regularization.

This grid search was done for $n_{epochs} \in \{15, 20, 25\}$, and $\lambda \in \{10^{-5}, 10^{-3}, 10^{-1}\}$. The resulting cross-validated accuracies and standard error can be found in figure 6: Here accuracies are distributed around a

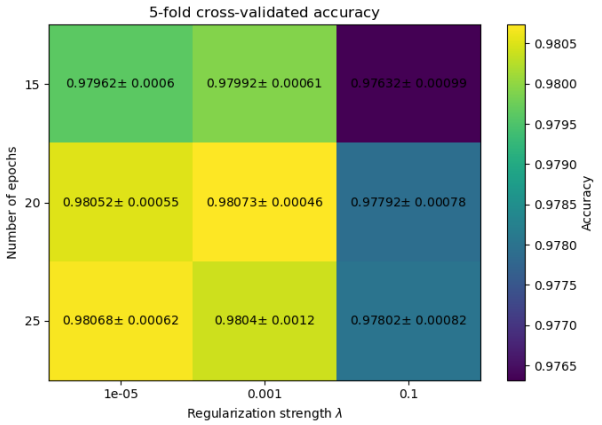


Figure 6. Heatmap of cross-validated accuracies over number of epochs in training and regularization strength λ. 5-fold cross-validation conducted on model with 512 hidden nodes and a learning rate of $\gamma = 10^{-3}$. Highest accuracy was achieved for $(\lambda = 10^{-3}, n = 20)$.

very similar mean with very little spread. So much so that we have to look at scores all the way down to differences that are smaller than their standard errors (see the four scores with $20 - 25$ epochs with the two smallest λ values). When this happens, the one-standard-error rule must be applied. We select the simplest model among those with the highest scores indistinguishable to one standard error. In this case, the simplest model is actually the one with the highest accuracy in the grid-search, with 20 epochs and a regularization strength of 10^{-3}. Simplicity prefers less training, and a stricter regularization which keeps the model from overfitting. This is our 5-fold cross-validated best model on the training data, and we will move forward with the parameters found in table II

Parameter	Value
γ	10^{-3}
n_{epochs}	20
$n_{\text{hidden nodes}}$	512
λ	10^{-3}

Table II. Best parameters for MLP with one hidden layer with ReLU activation. Found by optimizing for accuracy through 5-fold cross-validation and use of the one-standard-error rule on the MNIST handwritten digits training set.

After the model parameters were decided, the only thing left to do with our model was to train it on the entire training set and to score it on the hereto untouched test set. In addition to our own MLP with one hidden layer we trained two baseline models our network should be able to outperform, and an instance of sklearn's MLPClassifier with identical parameters to that of our own network.

Any classification model should be able to beat, not just 0 accuracy, but a model that simply always predicts the most frequent class in the training set. For the MNIST set, this class was the digit 1. So the first baseline was a model that always guesses the digit 1. An MLP with one hidden layer should also be able to beat a logistic regression model, as a logistic regression model is simply an MLP without a hidden layer. We used scikitlearn's LogisticRegression model with the *saga* solver, well adapted to large datasets such as MNIST. The accuracies of the four models can be found in table III We see that

Model	Accuracy	95% CI	Time to fit
Guess only 1	0.114 ± 0.003	$0.107 - 0.112$	0s
Logistic	0.926 ± 0.003	$0.920 - 0.931$	20m 24s
Our MLP	0.981 ± 0.001	$0.979 - 0.984$	1m 20s
Sklearn MLP	0.984 ± 0.001	$0.981 - 0.986$	1m 35s

Table III. Test-set accuracy, 95% confidence interval (calculated using eq.22), and time to fit for the two baselines and the final MLP with cross-validated optimized parameters. Both with our NeuralNetwork class, and sklearn's MLPClassifier

our model indeed does beat both the "guess only 1" model and the logistic regression model far outside its 95% confidence interval. The time to fit was also a lot lower than for that of the logistic regression saga solver. Our network also performs in the same range as Sklearn's MLPClassifier. Their 95% confidence intervals do overlap.

We can also take a look at our model's confusion matrix to see exactly where the errors it makes lie. The confusion matrix can be found in figure 7: We can see an almost diagonal matrix, as we would expect from a model with such high accuracy. One thing to notice is that a large portion of the model's errors are concentrated on just a few matrix elements. Out of all the 188 errors 21% of them are divided amongst just 4 squares. That being predicting 2 to be 8, 5 to be 3, 7 to be 9, and 4 to be 9. If we take a look at these mistakes, we realize that they actually arise from the number being similar in physical form. When you factor in individual handwriting, and the grainyness of the pictures, I am sure that humans might make similar mistakes. Haven't we all gotten the wrong answer on a math problem some time because we have confused two numbers in our own sloppy handwriting? Another thing to note is that, in breaking with our expectations, the matrix

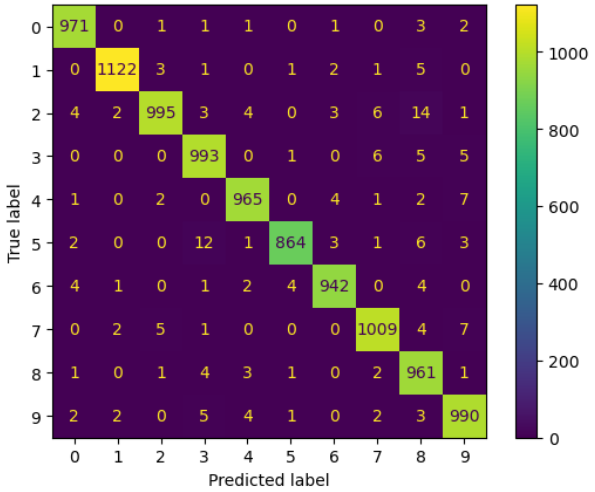


Figure 7. Confusion matrix of our EstimatorClassifier MLP on test set with parameters from table II.

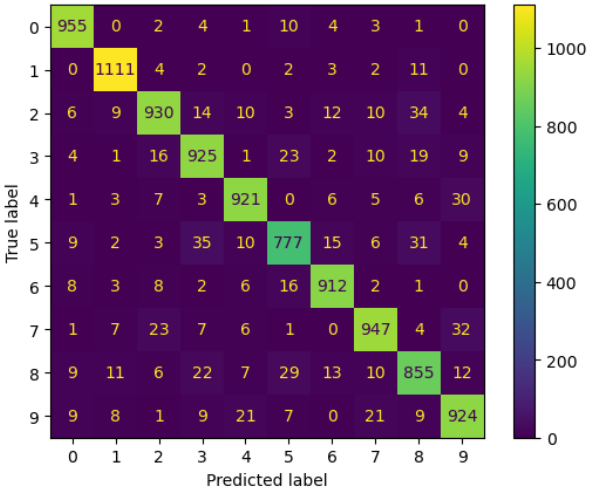


Figure 8. Confusion matrix of logistic regression model with saga solver on test set.

is not even close to being symmetric. 14 predictions that the number 2 is the number 8, but only 1 confusion the other way around? It isn't that the numbers are too similar, but simply that some feature that the numbers 2 and 8 share has been associated too strongly with the number 8. Most confusions go one way OR the other. This isn't random noise, but actual model bias. If we compare this to the confusion matrix of the logistic regression model found in figure 8: We see that this matrix is much closer to being symmetric. It confuses 5 with 3, but also 3 with 5. Its mistakes are most likely simply model limitations. We have not done a bias variance tradeoff analysis in this project, but based on the confusion matrices it should be apparent that the error of the neural net is in large part bias, and the error of the logistic model is in large part variance.

V. CONCLUSION

Our neural network managed to fit the Runge function with noise floor 0.01, obtaining an $MSE =$ comparable to that of a linear OLS model. Stochastic gradient descent with use of Adam optimization was found to be the most effective learning process. The width and the depth of the network had to stay lean and shallow with only 1 hidden layer and between 10-25 nodes to avoid overfitting. However, when using L_1 or L_2 regularization, overfitting could be avoided such that the complexity of the network could increase while MSE decreased. For our classification model, we conducted a 5-fold cross-validated parameter selection for a multilayer perceptron with one hidden layer and ReLU activation on the MNIST handwritten digits training set. The parameters we selected for were learning rate, number of epochs, hidden layer width, and L^2 regularization strength. We found, using grid-searches and the one-standard-error rule, that the optimal parameters were: $\gamma = 10^{-3}, n_{epochs} = 20, n_{\text{hidden nodes}} = 512, \lambda = 10^{-3}$. The network achieved an accuracy of $98.1\% \pm 0.1\%$ on the test set. Our comparison with logistic regression shows that one hidden layer makes a significant difference, and that a MLP is better at achieving higher accuracies on this classification problem, even with just one hidden layer.

[1] M. Hjort-Jensen, *Machine Learning and the Physical Sciences From Discriminative methods to generative methods and reinforcement learning* (Springer, 2026).
[2] OpenStax, "A confidence interval for a population proportion," Section 8.3.
[3] M. Nielsen, "Neural networks and deep learning," http://neuralnetworksanddeeplearning.com/chap2.html (2019), chapter 2. Accessed online: Nov 2026.
[4] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," (2018).
[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, Journal of Machine Learning Research **12**, 2825 (2011).
[6] Y. LeCun, C. Cortes, C. Burges, *et al.*, Florham Park, NJ, USA (2010).

Appendix A: Use of LLM tools

1. Tools used

Tool	Accessed
GPT 5.6 Terra	UiO GPT nov. 2026
Claude Sonnet 5.5	Cursor nov. 2026
Grok 4.6	Cursor nov. 2026

Table IV. LLM Tools

Section	Level	Notes
Abstract	0	
Introduction	0	
Theory/methods	0	
Code, implementation, tests	2	Debugging and plotting
Results and discussion	0	
Conclusion	0	

Table V. LLM assistance

2. How tools were used

LLM's was mainly used in this project to write code for plotting and visualization.